ITI Examination System

The ITI Examination System is a database-driven system for managing exams, questions bank and student assessments.

Project Agenda

1- Youssef Wagih 2- Ziad Tamer 3- Almotasem Bellah

4- Ziad Hany 5-Esraa Mohamed

Supervised By: ENG. Rami

Project Agenda

1. Introduction
2. System Objectives
3. System Scope
4. Database Architecture
5. Database Schema Overview
6. Entity Relationship Diagram (ERD)
7. Database Dictionary
8. Stored Procedures Overview
    - 8.1 Data Management Procedures
    - 8.2 Exam Procedures
9. Stored Procedures Logical Flow
    - 9.1 Exam Generation
    - 9.2 Student Exam Attempt
    - 9.3 Exam Correction
10. Reports
11. Conclusion

1. Introduction

The ITI Examination System is a database-driven application designed to automate the process of creating, managing, and correcting examinations. The system supports academic tracks, courses, question banks, exams, and student evaluation. It aims to reduce manual effort, improve accuracy, and ensure consistency in the examination and grading process.

2. System Objective

The main objectives of this system are to:

- Automate exam generation based on predefined rules
- Maintain a centralized question bank
- Store model answers and student responses securely
- Perform automatic exam correction
- Generate accurate student performance reports

3. System Scope

The system covers the management of academic tracks and courses, question creation and categorization, exam generation, student exam participation, answer storage, and grading. It supports multiple question types, including Multiple Choice Questions (MCQ) and True/False questions.

4. Database Architecture

The system is built around a centralized SQL Server database named **ITI_ExamSystem**. Business logic is implemented using stored procedures to ensure performance, security, and data integrity. All data operations such as exam generation, answer submission, and grading are handled at the database level.

5. Database Schema Overview

The database schema consists of several related entities that represent the academic structure and examination workflow. Core entities include Department, Track, Course, Topic, Student, Instructor, Question, Choice, Exam, StudentExam, ExamQuestion, InstructorCourse and StudentAnswer. These entities work together to support the complete examination lifecycle.

6. Entity Relationship Diagram (ERD)

The ERD illustrates the structure of the ITI Examination System database and the relationships between its main entities. The system is centered around academic departments, tracks, courses, exams, questions, and student participation.

Each **Department** contains multiple tracks and each **track** can include multiple **courses**, and each course may be taught by one or more **instructors**. Courses contain several **topics**, which are used to categorize questions. Every course has a collection of **questions**, where each question is defined by its type (MCQ or True/False), difficulty level, grade, and correct answer.

For MCQ questions, multiple **choices** are stored, with one marked as the correct answer. Questions are associated with **model answers**, which are used during automatic exam correction.

An **exam** is generated for a specific course and consists of a selected set of questions. The relationship between exams and questions is many-to-many, allowing questions to be reused across different exams. Students are assigned to exams through a **StudentExam** relationship, which records exam participation.

Students submit their responses through **StudentAnswer**, which links each student to their selected answer for every question in the exam. These answers are compared with the model answers to calculate the final grade.

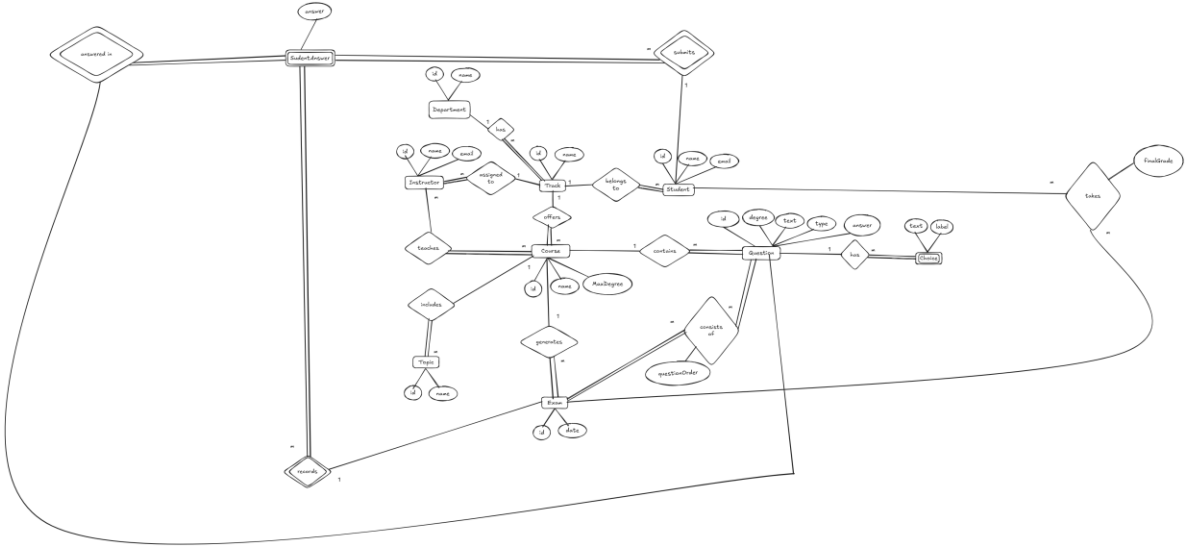

Figure 1: Entity Relationship Diagram of the ITI Examination System

7. Database Dictionary

1. **Department**
    a. Purpose: Stores academic department that contains multiple tracks.
    b. Columns:
        i. deptID: (INT) – Unique identifier for the department
        ii. deptName: (VARCHAR) – The department name
    c. Primary Key:
        i. deptID
    d. Foreign Keys:
        i. None
    e. Constraints:
        i. deptName is unique for each department and not allow null
2. **Track**
    a. Purpose: Stores academic tracks that group related courses, instructors, and students.
    b. Columns:
        i. TrackId (INT) – Unique identifier for the track
        ii. TrackName (VARCHAR) – Name of the track
        iii. deptID (INT) – assigned to department
    c. Primary Key:
        i. TrackId
    d. Foreign Keys:
        i. deptID in Department table
    e. Constraints:
        i. TrackName is unique for each track and Not allow null
3. **Student**
    a. Purpose: Stores student information and their assigned academic track.
    b. Columns:
        i. StudentId (INT) – Unique student identifier
        ii. StudentName (VARCHAR) – Student full name
        iii. Email (VARCHAR) – Student email
        iv. TrackId (INT) – Assigned track
    c. Primary Key:
        i. StudentId
    d. Foreign Keys:
        i. TrackId → Track(TrackId)
    e. Constraints:
        i. Email must be unique
        ii. Student must belong to exactly one track

7. Database Dictionary cont'd

4. **Instructor**
    a. Purpose: Stores information about instructors assigned to tracks and courses.
    b. Columns:
        i. InstructorId (INT) – Unique identifier for instructor – PK
        ii. InstructorName (VARCHAR) – Instructor full name
        iii. Email (VARCHAR) – Instructor email – should be Unique for each instructor
        iv. TrackId (INT) – Assigned track –> Track(TrackId)
    c. Primary Key:
        i. InstructorId
    d. Foreign Keys:
        i. TrackId → Track(TrackId)
    e. Constraints:
        i. Email must be unique
        ii. Instructor must belong to one track
5. **Course**
    a. Purpose: Stores courses offered under each academic track.
    b. Columns:
        i. CourseId (INT) – Unique course identifier
        ii. CourseName (VARCHAR) – Course name
        iii. MaxDegree (INT) – Maximum exam degree
        iv. TrackId (INT) – Related track
    c. Primary Key:
        i. CourseId
    d. Foreign Keys:
        i. TrackId → Track(TrackId)
    e. Constraints:
        i. Course must belong to one track
        ii. Maximum degree must be positive (if you have max value edit it)
6. **InstructorCourse**
    a. Purpose: Stores Assigned Instructors to their courses.
    b. Columns:
        i. InstructorID (INT) – Instructor ID
        ii. CourseID (INT) – Course ID
    c. Primary Key:
        i. composed key (CourseId, InstructorID)
    d. Foreign Keys:
        i. CourseId → Course(CourseId)
        ii. InstructorID –> Instructor (InstructorID)

7. Database Dictionary cont'd

7. **Topic**
    a. Purpose: Stores topics included within each course to organize questions.
    b. Columns:
        i. TopicId (INT) – Unique topic identifier
        ii. TopicName (VARCHAR) – Topic name
        iii. CourseId (INT) – Related course
    c. Primary Key:
        i. TopicId
    d. Foreign Keys:
        i. CourseId → Course(CourseId)
    e. Constraints:
        i. Topic must belong to one course
        ii. TopicName Not allow null
8. **Question**
    a. Purpose: Stores course questions with their type, difficulty, and grading details.
    b. Columns:
        i. QuestionId (INT) – Unique question identifier
        ii. QuestionText (VARCHAR) – Question content
        iii. QuestionType (VARCHAR) – MCQ or True/False
        iv. QuestionAnswer (VARCHAR) – correct answer for question
        v. Grade (INT) – Question grade and difficulty level
            - It represents both the difficulty level and the scoring weight of the question at the same time
        vi. CourseId (INT) – Related course
        vii. Note
            - For questions of type True/False (TF), the model answer (QAnswer) stores:
            - T → True
            - F → False
            -
            - During exam display and reporting, the system treats:
            - A → True
            - B → False
            - as the available answer choices.
    c. Primary Key:
        i. QuestionId
    d. Foreign Keys:
        i. CourseId → Course(CourseId)
    e. Constraints:
        i. Question type must be valid (MCQ or TF)
        ii. Grade must be between 1 and 3 (1 → easy , 2 → medium, 3 → hard)

iii. QuestionText could not be empty
iv. QuestionAnswer should be either (a, b, c) if question type MCQ or (T, F) if TF

7. Database Dictionary cont'd

9. Choice

a. Purpose: Stores possible choices for MCQ questions.
b. Columns:
   i. ChoiceLabel (VARCHAR) – A or B or C label for the choice
   ii. ChoiceText (VARCHAR) – Choice value
   iii. QuestionId (INT) – Related question
c. Primary Key:
   i. Composed Key (QuestionId , ChoiceLabel)
d. Foreign Keys:
   i. QuestionId → Question(QuestionId)
e. Constraints:
   i. ChoiceLabel should be A or B or C
   ii. ChoiceText could not be empty text
   iii. ChoiceText should be unique per question
f. Note:
   - For True/False (TF) questions, no records are stored in the Choice table.
   - The possible answers are implicitly defined as:
   - A → True
   - B → False
   - These values are handled logically in stored procedures

10. Exam

a. Purpose: Stores generated exams for courses.
b. Columns:
   i. ExamId (INT) – Unique exam identifier
   ii. ExamDate (DATETIME) – Exam date and default date is the exam generation date
   iii. CourseId (INT) – Related course
c. Primary Key:
   i. ExamId

d. Foreign Keys:
    i. CourseId → Course(CourseId)
e. Constraints:
    i. Exam must be linked to one course

11. ExamQuestion

a. Purpose: Links exams to their selected questions.
b. Columns:
    i. ExamId (INT) – Related exam
    ii. QuestionId (INT) – Related question
    iii. QuestionOrder (INT) – Order of questions in exam
c. Primary Key: Composite (ExamId, QuestionId)
d. Foreign Keys:
    i. ExamId → Exam(ExamId)
    ii. QuestionId → Question(QuestionId)
e. Constraints: Question order must be unique per exam

7. Database Dictionary cont'd

12. StudentExam

a. Purpose: Records student participation in exams.
b. Columns:
    i. StudentId (INT) – Related student
    ii. ExamId (INT) – Related exam
    iii. FinalGrade (DECIMAL(5,2)) – Percentage grade
c. Primary Key:
    i. composed key (StudentId, ExamId)
d. Foreign Keys:
    i. StudentId → Student(StudentId)
    ii. ExamId → Exam(ExamId)
e. Constraints:
    i. Student can register for an exam only once

13. StudentAnswer

a. Purpose: Stores answers submitted by students during exams.
b. Columns:
    i. studentId (INT) – Related id for student
    ii. ExamId (INT) – Related exam attempt
    iii. QuestionId (INT) – Related question
    iv. Answer (VARCHAR) – Student answer
c. Primary Key:

    i. composed key (studentId , ExamId, questionId)
d. Foreign Keys:
    i. (StudentID , ExamId) → StudentExam(StudentId, ExamId)
    ii. QuestionId → Question(QuestionId)
e. Constraints:
    i. One answer per question per exam attempt

8. Stored Procedures Overview

8.1 Data Management Procedures:

These procedures manage (DML Statements) core data such as Departments, tracks, courses, questions, choices, Topics, students, and instructors. They ensure validation and prevent inconsistent or duplicate data.

8.2 Exam Procedures:

These procedures handle exam generation, student registration, answer submission, and exam correction. They form the backbone of the system's business logic.

9. Exam Stored Procedures Logical Flow

9.1 Exam Generation Stored Procedure

Purpose:

Generates a new exam for a specific course by randomly selecting a fixed number of True/False and MCQ questions.

Logical Steps (Procedure Flow)

1. Receive the course name, number of True/False questions, number of MCQ questions, and an output exam ID.
2. Validate that the total number of requested questions equals the required exam size.
3. Retrieve the course identifier based on the provided course name.
4. Validate that the course exists in the system.
5. Generate a new unique exam identifier.
6. Create a new exam record linked to the selected course with the current date.
7. Randomly select the required number of True/False questions for the course.
8. Assign an order number to each selected True/False question and link them to the exam.
9. Validate that the required number of True/False questions is available.
10. Randomly select the required number of MCQ questions for the same course.
11. Assign sequential order numbers to the MCQ questions following the True/False questions.
12. Validate that the required number of MCQ questions is available.
13. Save all exam and question assignments as a single transaction.
14. If any validation or insertion fails, cancel the operation and roll back all changes.
15. Return the generated exam ID.

9. Exam Stored Procedures Logical Flow

9.2 Student Exam Attempt

Purpose:

Stores a student's answers for a specific exam while ensuring exam validity, student eligibility, and preventing duplicate submissions.

Logical Steps (Procedure Flow)

1. Receive the exam identifier, student name, and the student's answers for the ten exam questions.
2. Retrieve the student identifier based on the provided student name.
3. Validate that the student exists in the system.
4. Validate that the specified exam exists.
5. Confirm that the student is registered for the specified exam.
6. Check that the student has not already submitted answers for this exam.
7. Verify that the exam contains exactly ten questions.
8. Match each submitted answer to its corresponding question based on the question order in the exam.
9. Store the student's answers for all exam questions.
10. If any validation fails, stop the process and return an error without saving any data.

9.3 Exam Correction And The Final Grade

Purpose:

Calculates the final grade of a student for a specific exam by comparing the student's answers with the correct answers and storing the result.

Logical Steps (Procedure Flow)

1. Receive the exam identifier and the student name.
2. Retrieve the student identifier based on the provided student name.
3. Validate that the student exists in the system.
4. Verify that the student has submitted answers for the specified exam.
5. Calculate the total possible degree of the exam based on the grades of all exam questions.
6. Calculate the student's earned degree by summing the grades of correctly answered questions.
7. Handle cases where the student has no correct answers by assigning a zero score.
8. Calculate the final grade as a percentage of the total exam degree.
9. Store the calculated final grade for the student's exam record.
10. Return the final grade as the procedure output.

10. Reports

10.1 Exam_GetQuestionsAndChoices

Purpose:

Retrieves all questions of a specific exam along with their details and available choices, providing a complete view of the exam content.

Logical Steps (Procedure Flow)

1. Receive the exam identifier.
2. Retrieve all questions assigned to the specified exam in their defined order.
3. For each question, return its basic information, including:
    - Question text
    - Question type
    - Question degree
    - Correct answer
4. Retrieve all available choices for MCQ questions associated with the exam.
5. Include questions without choices (True/False) by returning them without choice data.
6. Present the result ordered by the question sequence in the exam, followed by the choice order.

10.2 Course_GetTopics

Purpose:

Retrieves all topics associated with a specific course.

Logical Steps (Procedure Flow)

1. Receive the course identifier.
2. Validate the existence of the specified course implicitly through data retrieval.
3. Retrieve all topics that belong to the given course.
4. For each topic, return its identifier and name along with the related course information.
5. Sort the topics in ascending order based on their identifiers.
6. Return the result set for display or reporting purposes.

10. Reports

10.3 StudentExam_GetAnswers report

Purpose:

Retrieves a detailed view of a student's answers for a specific exam, including question details, correct answers, student answers, and earned degrees.

Logical Steps (Procedure Flow)

1. Receive the student identifier and the exam identifier.
2. Retrieve all questions assigned to the specified exam in their defined order.
3. For each question, return its basic information including:
    a. Question text
    b. Question type
    c. Question degree
    d. Correct answer
4. Retrieve the student's submitted answer for each exam question, if available.
5. Compare the student's answer with the correct answer to determine correctness.
6. Calculate the earned degree for each question:
    a. For True/False questions, assign the full question degree for a correct answer and zero for an incorrect answer.
    b. For MCQ questions, assign the full question degree for a correct answer and zero otherwise.
7. Include all available choices for MCQ questions, when applicable.
8. Present the results ordered by question sequence in the exam, followed by choice order.

10. Reports

10.4 Student Within a Department report

Purpose:
Retrieves a list of students belonging to a specific department, including their personal information, assigned track, and department details.

Logical Steps (Procedure Flow)

1. Receive the department identifier as input.
2. Validate that the specified department exists in the system.
3. If the department does not exist, stop the process and return an error message.
4. Retrieve all students whose tracks belong to the specified department.
5. For each student, retrieve the following information:
    a. Student identifier
    b. Student name
    c. Student email
    d. Track name
    e. Department name
6. Return the result set for reporting purposes.

10.5 Student Grades report

Purpose:
Retrieves the exam grades of a specific student, including course details, final grade, maximum degree, and exam percentage.

Logical Steps (Procedure Flow)

1. Receive the student identifier as input.
2. Validate that the specified student exists in the system.
3. If the student does not exist, stop the process and return an error message.
4. Retrieve all exams taken by the specified student.
5. For each exam, retrieve the related course information.

6. Return the following details for each course exam:
    a. Student identifier
    b. Student name and email
    c. Course name
    d. Final exam grade
    e. Maximum course degree
    f. Exam percentage calculated from the final grade and maximum degree
7. Return the result set for reporting purposes.

10.6 Instructor Courses report

Purpose:

Generates a report showing the courses taught by a specific instructor along with the number of students enrolled in each course.

Logical Steps (Procedure Flow)

1. Receive the instructor identifier as input.
2. Validate that the specified instructor exists in the system.
3. If the instructor does not exist, stop the process and return an error message.
4. Retrieve all courses assigned to the specified instructor.
5. For each course, identify students enrolled through the related academic track.
6. Count the number of students enrolled in each course.
7. Return the instructor name, course name, and total number of students per course.

10. Conclusion

Summary of System Capabilities

The ITI Examination System provides a comprehensive and integrated platform for managing academic examinations. The system supports the organization of tracks, departments, courses, instructors, and students, while enabling structured exam creation, question management, student participation, and automated grading. It allows accurate tracking of student performance, detailed reporting, and secure storage of examination data, ensuring efficiency and consistency throughout the examination lifecycle.

Benefits of Database Centric Design

Adopting a database-centric architecture enhances the system's reliability, scalability, and performance. Core business logic is implemented through stored procedures, which ensures data integrity, reduces redundancy, and enforces consistent validation rules. This approach minimizes dependency on application-level logic, improves execution speed, enhances security, and simplifies maintenance by centralizing data processing within the database engine.

Project Outcomes

This project successfully delivers a robust examination management solution that meets academic and administrative requirements. The system demonstrates effective database design, efficient query optimization, and practical use of stored procedures for reporting and automation. Overall, the project reflects a strong understanding of relational database concepts and provides a scalable foundation that can be extended to support additional academic features in the future.